

Advanced Database Management System Lab

Final Project Report



Session: Fall 2024

Submitted By:

Muhammad Ayan Sajid	2024-CS-661
Muhammad Bilal	2024-CS-657
Shareen Asim	2024-CS-693
Muhammad Husnain	2024-CS-696

Submitted To:

Ms. Rida Saeed

Department of Computer Science, UET Lahore, New Campus

Table of Contents

Abstract	4
1 Introduction.....	4
2 Theoretical Framework and Related Work	5
2.1 Task 1: Distributed Systems and the CAP Theorem.....	5
2.2 Task 2: Replica Set Consensus	5
2.3 Task 3: Indexing Complexity Models.....	5
2.4 Task 4: Query Execution Plans and Relational Algebra	6
2.5 Task 5: Concurrency Isolation and Deadlock Cycles	6
2.6 Task 6: OLAP Warehousing and Pattern Extraction.....	6
3 System Architectures & Simulation Methodologies.....	6
3.1 Task 1 Simulation: Distributed Node Partitioning.....	6
3.2 Task 2 Deployment: Three-Node Local Replica Set	7
3.3 Task 3 Implementation: Memory-Resident Index Simulators	7
3.4 Task 4 Optimization: Schema & Query Re-Engineering.....	7
3.5 Task 5 Simulation: Waited Conflict Lock Manager.....	8
3.6 Task 6 Framework: Star Schema Deployment & Extraction.....	8
4 Empirical Evaluations & Experimental Results	8
4.1 Task 1 Evaluation: CAP Partition Trade-offs.....	8
4.2 Task 2 Evaluation: MongoDB Failover Latency	9
4.3 Task 3 Evaluation: Indexing Performance Profiling.....	12
4.4 Task 4 Evaluation: Relational Execution Plans & page I/O	12
4.5 Task 5 Evaluation: Deadlock Lock Analysis	14
4.6 Task 6 Evaluation: Star Schema Mining Analysis.....	15
5 Technical Challenges & Resolution Strategies	19
5.1 Task 2 Challenge: Port Binding Conflicts with Default Services.....	19
5.2 Task 6 Challenge: SQL Server 'Invalid Object Name' Context Error.....	19
5.3 Task 6 Challenge: Pandas SQL Connection Warnings	20
6 Conclusion & Future Work.....	20

Table of Figures

Figure 1 CP Mode simulation demonstrating active transaction blocking and write rejection during a majority quorum loss under network partitioning.	9
Figure 2 AP Mode simulation illustrating concurrent data divergence (stale read on the isolated Node 3 versus updated read on Node 1) while a partition is active.	9
Figure 3 Network healing and asynchronous reconciliation phase showing the transition of the recovered Node 3 to eventual consistency.	9
Figure 4 Console output showing rs.status() output indicating node 27018 as offline and node 27019 as primary.....	11
Figure 5 : Performance execution times comparing tree-based range scans vs. hash table full scans.....	12
Figure 6 SQL Server I/O and execution time statistics for the unoptimized query, demonstrating a high CPU utilization of 165,953 ms.	13
Figure 7 SQL Server I/O and execution time statistics for the optimized query, showing a reduction in CPU utilization to 78 ms.....	13
Figure 8 Physical execution plan of the unoptimized query in SSMS, showing the nested loop joins, high-cost table scanning operators, and a intermediate Index Spool (Eager Spool) bottleneck.....	14
Figure 9 Physical execution plan of the optimized query in SSMS, showing the streamlined set-based Hash Match join and aggregation operators that replaced the nested loops.	14
Figure 10 Waits-For Graph (WFG) showing circular locking dependencies and automated transaction rollback.....	15
Figure 11 Concurrency simulation console output displaying lock acquisition, mutual lock-conflict blocking, deadlock detection, and subsequent rollback/release phases.....	15
Figure 12 SQL query executed within the ETL pipeline to retrieve transactional datasets by joining the central fact table with the product dimension table.	16
Figure 13 SQL query executed within the ETL pipeline to aggregate customer spending metrics and retrieve qualitative demographic attributes from the customer dimension table.	16
Figure 14 Entity-Relationship Diagram (ERD) of the analytical database, illustrating the de-normalized Star Schema with a central fact table (fact_sales) linked to surrounding dimension tables.	17
Figure 15 Console output of the Apriori data mining execution showing the one-hot encoded matrix, extracted frequent itemsets, and computed association rules (support, confidence, and lift).....	18
Figure 16 Console output of the Decision Tree classification model showing the aggregated customer profile targets, the single-leaf decision rule, and the premium buyer plan prediction. .	19

Abstract

This report presents an end-to-end empirical study evaluating the architectural trade-offs, optimization techniques, and analytical capabilities of modern database management systems (DBMS). Through a series of decoupled software simulations and physical deployments, we evaluate: (i) distributed consistency under network partitions (CAP theorem), (ii) Raft-like primary-secondary replication and automated failover mechanics, (iii) algorithmic scaling of B+ Tree and Hash-based indexing, (iv) physical operator transitions in relational query execution plans, (v) transactional deadlocks under Two-Phase Locking (2PL), and (vi) multi-dimensional mining on a Star Schema data warehouse.

Our empirical evaluations demonstrate that query restructuring combined with targeted non-clustered indexing reduced relational CPU computation time by a factor of 2,127 and logical I/O page reads by a factor of 1,993. Algorithmic profiling confirmed that while Hash indexes achieve constant-time $O(1)$ lookup for exact matches, B+ Trees outperform them in range scans by avoiding full table scans. Finally, we demonstrate that data mining pipelines executed directly on a structured Star Schema warehouse yield significant cross-selling rules and predictable classification boundaries.

1 Introduction

Modern enterprise database architectures must scale horizontally to manage highly concurrent workloads while preserving data integrity and high availability [1, 2]. These strict requirements present database engineers with complex architectural trade-offs. The design of distributed, transactional, and analytical database systems is governed by physical constraints, ranging from network partition failures to hardware I/O limitations.

This report systematically evaluates these database engineering principles through six distinct experimental phases:

1. **Distributed Consensus and Partitioning (Task 1):** Simulating the fundamental trade-offs between Consistency and Availability (CAP theorem) under active network partition events.
2. **High Availability Replication (Task 2):** Deploying a physical three-node MongoDB replica set on customized ports to evaluate data replication and primary-failover election speeds.
3. **Algorithmic Indexing Performance (Task 3):** Benchmarking point-lookup and range-query execution times on B+ Tree and Hash-based index models using a randomized 50,000-record dataset.

4. **Relational Query Optimization (Task 4):** Restructuring an inefficient, correlated subquery on Microsoft SQL Server to analyze execution plans, join algorithms, and database page I/O.
5. **Transactional Concurrency and Isolation (Task 5):** Simulating lock contention and deadlock detection using a Two-Phase Locking (2PL) protocol and Waits-For Graph (WFG) analysis.
6. **Data Warehousing & Analytical Mining (Task 6):** Designing a Star Schema in SQL Server and extracting data to run Apriori association mining and Decision Tree classification.

2 Theoretical Framework and Related Work

2.1 Task 1: Distributed Systems and the CAP Theorem

Eric Brewer's CAP Theorem states that a partitioned distributed system can guarantee either Consistency (CP) or Availability (AP), but not both simultaneously. In a CP system, write operations are blocked if a majority quorum cannot be reached, ensuring that clients never read stale or conflicting data. In an AP system, every reachable node accepts local writes and serves reads, allowing temporary data divergence that must be resolved asynchronously (e.g., using a Last-Write-Wins protocol) when the partition heals.

2.2 Task 2: Replica Set Consensus

High availability in modern document stores (such as MongoDB) relies on active replication across multiple nodes. The nodes communicate via heartbeat signals every 2 seconds. If the primary node stops responding, the secondary nodes run an election using a consensus protocol to promote a new primary. This process ensures automatic failover without requiring manual configuration at the application layer.

2.3 Task 3: Indexing Complexity Models

The choice of database index directly impacts query latency:

- **Hash Indexes:** Use a hash function to map keys to bucket arrays. They achieve optimal point-query complexity ($O(1)$) but perform poorly on range queries ($O(N)$) because hashing does not preserve sequential order.
- **B+ Trees:** Balanced search trees that store routing keys in internal nodes and actual data pointers in sorted, linked leaf nodes. They resolve exact lookups in $O(\log N)$ time and execute range queries efficiently ($O(\log N + K)$) by traversing the linked leaf nodes sequentially.

2.4 Task 4: Query Execution Plans and Relational Algebra

Relational Database Engines compile declarative SQL queries into physical execution plans. Inefficient queries, such as those with correlated subqueries inside a select projection list, force the engine to use a nested-loop join. This results in $O(N \times M)$ complexity and high CPU usage. Restructuring these queries into explicit set-based joins allows the optimizer to use efficient Hash Match or Merge Join operators, which reduces disk and memory page reads.

2.5 Task 5: Concurrency Isolation and Deadlock Cycles

Transaction isolation in relational systems is maintained using locking protocols. Under Two-Phase Locking (2PL), transactions must acquire all needed locks during a growing phase and release them during a shrinking phase. While 2PL ensures serializability, it introduces the risk of deadlocks when transactions acquire locks on resources in a conflicting order. These deadlocks are modeled using directed Waits-For Graphs (WFG), where a cycle indicates a deadlock state that must be resolved by aborting a transaction.

2.6 Task 6: OLAP Warehousing and Pattern Extraction

Transactional databases (OLTP) are optimized for fast writes, whereas analytical systems (OLAP) are structured as Star Schemas to speed up read-heavy queries. A central Fact Table stores numerical metrics, surrounded by descriptive Dimension Tables. Once the OLAP data is joined and extracted, data mining techniques like the Apriori Algorithm (using support, confidence, and lift) and Decision Trees can be applied to uncover purchasing patterns and classify customer segments.

3 System Architectures & Simulation Methodologies

3.1 Task 1 Simulation: Distributed Node Partitioning

The CAP simulation was built as a modular object-oriented Python pipeline containing three distinct node objects. The system state is managed by a centralized network manager that can toggle a partition flag. When the partition is active, Node 3 is isolated from the coordinator nodes (Node 1 and Node 2).

The CP implementation enforces majority quorum checks on every write, rejecting any updates if a node is unreachable. The AP implementation accepts writes on any reachable node and uses physical write timestamps to resolve conflicts via a Last-Write-Wins (LWW) reconciliation engine when the network partition heals.

3.2 Task 2 Deployment: Three-Node Local Replica Set

The high availability replication cluster was deployed using three distinct MongoDB 8.2 processes running on a single local host. To bypass conflicts with pre-existing Windows services on the default port 27017, the instances were bound to customized ports:

- **Node 1 (Primary Prospect):** Port 27018 | dbPath: Task 2a | priority: 2
- **Node 2 (Secondary Prospect):** Port 27019 | dbPath: Task 2b | priority: 1
- **Node 3 (Secondary Prospect):** Port 27020 | dbPath: Task 2c | priority: 1

The cluster was initialized from mongosh on port 27018 using the replica set identifier rs0.

3.3 Task 3 Implementation: Memory-Resident Index Simulators

To benchmark indexing performance, we implemented two memory-resident structures in Python:

- **BPlusTreeMockIndex:** Maintains key-value records in a sorted array. It uses binary search for point lookups and traverses keys sequentially for range queries, stopping as soon as the upper boundary is exceeded.
- **HashIndex:** Maps keys directly to values using Python's hash-map implementation. For range queries, it performs a full table scan over all stored keys.

The benchmark evaluated both structures against a dataset of 50,000 randomized records with keys distributed between 1 and 100,000.

3.4 Task 4 Optimization: Schema & Query Re-Engineering

The query optimization experiment was conducted in Microsoft SQL Server to evaluate the physical execution plans of unoptimized versus optimized SQL query variants on a dataset of **10,000 customers, 50,000 orders, and 150,000 order items**.

The initial unoptimized query utilized nested, correlated subqueries inside the select projection list:

```
select
    c.name,
    c.country,
    (select sum(o.total_amount) from orders o where o.customer_id = c.customer_id) as total_spent,
    (select avg(oi.price) from order_items oi where oi.order_id in (select o.order_id from orders o where o.customer_id = c.customer_id)) as avg_item_price
from customers c
where c.country = 'USA'
and (select sum(o.total_amount) from orders o where o.customer_id = c.customer_id) > 1000;
```

The optimized query variant consolidated the aggregates into a single pass using explicit set-based joins and non-clustered indexes:

```
select
    c.name,
    c.country,
    sum(o.total_amount) as total_spent,
    avg(oi.price) as avg_item_price
from customers c
inner join orders o on c.customer_id = o.customer_id
inner join order_items oi on o.order_id = oi.order_id
where c.country = 'USA'
group by c.customer_id, c.name, c.country
having sum(o.total_amount) > 1000;
```

3.5 Task 5 Simulation: Waited Conflict Lock Manager

To simulate database transaction concurrency, we built a centralized lock manager in Python. The lock manager registers exclusive write locks on shared data items (Account_A and Account_B). When a transaction attempts to acquire a lock held by another transaction, the manager registers a conflict and blocks the requesting thread.

The system includes a deadlock detection thread that monitors lock allocations. If a cycle is detected, the engine aborts the blocked transactions and releases their held locks to restore database consistency.

3.6 Task 6 Framework: Star Schema Deployment & Extraction

The analytical data warehouse was deployed on Microsoft SQL Server. The dimensional design features a central fact table (fact_sales) linked to three dimension tables (dim_customer, dim_product, dim_time) through foreign keys.

To run the data mining models, a Python ETL script connected to the database using `pyodbc`. It executed OLAP join queries to extract transactional shopping baskets and aggregated customer spend profiles into pandas dataframes.

4 Empirical Evaluations & Experimental Results

4.1 Task 1 Evaluation: CAP Partition Trade-offs

Executing the CAP simulation demonstrated the behavioral split between the CP and AP system configurations under network partitions:

- **CP Mode:** Writing "item_id" = "100" was successful during normal operations. When the partition was enabled (unreachable quorum), the CP system immediately rejected a write update to "200", preserving strict consistency at the expense of availability.
- **AP Mode:** The system successfully accepted the write update to "200" on the active nodes (Node 1 and Node 2) during the partition. Node 3 remained isolated and served the stale value "100" to clients, prioritizing system availability. When the partition healed, the LWW reconciliation engine successfully synchronized Node 3's value to "200".


```
CP Mode Demo
[CP Write Success] Key 'item_id' written to all nodes

Network Partition State: ACTIVE
[CP Write Fail] Cannot reach write quorum. Write rejected on item_id=200
[Network Info] Node 3 is isolated
[Read from Node 3] Key: 'item_id' -> Value: 100
```

Figure 1 CP Mode simulation demonstrating active transaction blocking and write rejection during a majority quorum loss under network partitioning.

```
AP Mode Demo
[AP Write Success] Key 'item_id' written to reachable nodes: [1, 2, 3]

Network Partition State: ACTIVE
[AP Write Success] Key 'item_id' written to reachable nodes: [1, 2]
[Network Info] Node 3 is isolated
[Read from Node 3] Key: 'item_id' -> Value: 100
[Read from Node 1] Key: 'item_id' -> Value: 200
```

Figure 2 AP Mode simulation illustrating concurrent data divergence (stale read on the isolated Node 3 versus updated read on Node 1) while a partition is active.

```
Healing network partition and reconciling data
Data synchronized across all nodes
[Read from Node 3] Key: 'item_id' -> Value: 200
```

Figure 3 Network healing and asynchronous reconciliation phase showing the transition of the recovered Node 3 to eventual consistency.

4.2 Task 2 Evaluation: MongoDB Failover Latency

The three-node replica set was initialized successfully on ports 27018, 27019, and 27020.

- **Data Replication:** Inserting a test document on the Primary node (port 27018) was automatically replicated to the Secondary node (port 27019). Setting the read preference to secondary allowed the record to be queried on the secondary node.
- **Automated Failover:** When the Primary node (port 27018) process was terminated, the surviving nodes detected the loss within their heartbeat windows. They initiated an election and promoted the secondary node on port 27019 to PRIMARY within seconds, restoring write availability to the cluster.

```

rs0 [direct: secondary] testdb> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-05-26T16:22:45.432Z'),
  myState: 1,
  term: Long('2'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    lastCommittedWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    appliedOpTime: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    durableOpTime: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    writtenOpTime: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    lastAppliedWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    lastDurableWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    lastWrittenWallTime: ISODate('2026-05-26T16:22:38.102Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1779812498, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'stepUpRequestSkipDryRun',
    lastElectionDate: ISODate('2026-05-26T16:21:18.008Z'),
    electionTerm: Long('2'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1779812477, i: 1 }), t: Long('1') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1779812477, i: 1 }), t: Long('1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1779812477, i: 1 }), t: Long('1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    priorPrimaryMemberId: 0,
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-05-26T16:21:18.064Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-05-26T16:21:18.093Z')
  },
  electionParticipantMetrics: {
    votedForCandidate: true,
    electionTerm: Long('1'),
    lastVoteDate: ISODate('2026-05-26T15:46:46.037Z'),
    electionCandidateMemberId: 0,
    voteReason: '',
    lastWrittenOpTimeAtElection: { ts: Timestamp({ t: 1779810394, i: 1 }), t: Long('-1') },
    maxWrittenOpTimeInSet: { ts: Timestamp({ t: 1779810394, i: 1 }), t: Long('-1') },
    lastAppliedOpTimeAtElection: { ts: Timestamp({ t: 1779810394, i: 1 }), t: Long('-1') },
    maxAppliedOpTimeInSet: { ts: Timestamp({ t: 1779810394, i: 1 }), t: Long('-1') },
    priorityAtElection: 1
  },
  members: [
    {

```

```

    _id: 0,
    name: 'localhost:27018',
    health: 0,
    state: 0,
    stateStr: '(not reachable/healthy)',
    uptime: 0,
    optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeWritten: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeDate: ISODate('1970-01-01T00:00:00.000Z'),
    optimeDurableDate: ISODate('1970-01-01T00:00:00.000Z'),
    optimeWrittenDate: ISODate('1970-01-01T00:00:00.000Z'),
    lastAppliedWallTime: ISODate('2026-05-26T16:21:28.075Z'),
    lastDurableWallTime: ISODate('2026-05-26T16:21:28.075Z'),
    lastWrittenWallTime: ISODate('2026-05-26T16:21:28.075Z'),
    lastHeartbeat: ISODate('2026-05-26T16:22:43.046Z'),
    lastHeartbeatRecv: ISODate('2026-05-26T16:21:32.571Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: 'Error connecting to localhost:27018 (127.0.0.1:27018) :: caused by :: onInvoke :: caused by :: No connection could be made because the target machine actively
refused it.',
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 2
  },
  {
    _id: 1,
    name: 'localhost:27019',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 2202,
    optime: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    optimeWritten: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    optimeDate: ISODate('2026-05-26T16:22:38.000Z'),
    optimeDurableDate: ISODate('2026-05-26T16:22:38.000Z'),
    optimeWrittenDate: ISODate('2026-05-26T16:22:38.000Z'),
    lastAppliedWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    lastDurableWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    lastWrittenWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1779812478, i: 1 }),
    electionDate: ISODate('2026-05-26T16:21:18.000Z'),
    configVersion: 1,
    configTerm: 2,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
    _id: 2,
    name: 'localhost:27020',
    health: 1,

```

```

    state: 2,
    stateStr: 'SECONDARY',
    uptime: 2170,
    optime: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    optimeWritten: { ts: Timestamp({ t: 1779812558, i: 1 }), t: Long('2') },
    optimeDate: ISODate('2026-05-26T16:22:38.000Z'),
    optimeDurableDate: ISODate('2026-05-26T16:22:38.000Z'),
    optimeWrittenDate: ISODate('2026-05-26T16:22:38.000Z'),
    lastAppliedWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    lastDurableWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    lastWrittenWallTime: ISODate('2026-05-26T16:22:38.102Z'),
    lastHeartbeat: ISODate('2026-05-26T16:22:44.225Z'),
    lastHeartbeatRecv: ISODate('2026-05-26T16:22:44.800Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27019',
    syncSourceId: 1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 2
  }
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1779812558, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA= ', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1779812558, i: 1 })
}

```

Figure 4 Console output showing rs.status() output indicating node 27018 as offline and node 27019 as primary

4.3 Task 3 Evaluation: Indexing Performance Profiling

Evaluating the indexing structures on the 50,000-record dataset highlighted the trade-offs between hash tables and sorted trees:

- **Point Queries (Exact Match):** The Hash index completed the point lookup in **0.00001570 seconds**, outperforming the B+ Tree index which took **0.00004180 seconds**. This confirms the theoretical constant-time $O(1)$ efficiency of hash tables compared to the $O(\log N)$ complexity of tree structures.
- **Range Queries (Keys 20,000 to 30,000):** The B+ Tree index resolved the range query in **0.00281450 seconds**, outperforming the Hash index which took **0.00299040 seconds**. This is because the B+ Tree index traverses sorted, linked leaf nodes sequentially and terminates early when keys exceed the range threshold, whereas the Hash index must scan every key in the dataset ($O(N)$).

```
Exact Match Lookup Time:
- B+ Tree: 0.00004180 seconds
- Hash Index: 0.00001570 seconds (Typically faster due to O(1) complexity)

Range Query (20000 to 30000) Time:
- B+ Tree: 0.00281450 seconds (Iterates sequentially from low bounds)
- Hash Index: 0.00299040 seconds (Must scan all keys due to lack of sequential storage)
```

Figure 5 : Performance execution times comparing tree-based range scans vs. hash table full scans

4.4 Task 4 Evaluation: Relational Execution Plans & page I/O

Executing the unoptimized query in SQL Server put a heavy load on system resources, requiring **165,953 ms of CPU execution time** and **179,807 ms of elapsed time** (nearly 3 minutes).

This high CPU utilization was caused by full table scans and nested-loop joins, which resulted in **1,265,555 logical reads** on the order_items table and **105,387 logical reads** on internal system worktables.

After applying non-clustered indexes and restructuring the query into set-based joins, CPU execution time dropped to **78 ms** (a 2,127x improvement). This optimization allowed the engine to use efficient Index Seeks and Hash Match joins, reducing logical reads on order_items to just **635** (a 1,993x reduction) and eliminating the need for temporary disk worktables.


```
SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
    CPU time = 547 ms,  elapsed time = 631 ms.

(1993 rows affected)
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0,
Table 'Worktable'. Scan count 1993, logical reads 105387, physical reads 0,
Table 'order_items'. Scan count 1993, logical reads 1265555, physical reads 0,
Table 'orders'. Scan count 3, logical reads 546, physical reads 0,
Table 'customers'. Scan count 1, logical reads 50, physical reads 0

(1 row affected)

SQL Server Execution Times:
    CPU time = 165953 ms,  elapsed time = 179807 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 0 ms.
```

Figure 6 SQL Server I/O and execution time statistics for the unoptimized query, demonstrating a high CPU utilization of 165,953 ms.

```
SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 2 ms.
SQL Server parse and compile time:
    CPU time = 47 ms,  elapsed time = 122 ms.

(3032 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0,
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0,
Table 'order_items'. Scan count 1, logical reads 635, physical reads 0,
Table 'orders'. Scan count 1, logical reads 182, physical reads 0,
Table 'customers'. Scan count 1, logical reads 50, physical reads 0

(1 row affected)

SQL Server Execution Times:
    CPU time = 78 ms,  elapsed time = 3983 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 0 ms.
```

Figure 7 SQL Server I/O and execution time statistics for the optimized query, showing a reduction in CPU utilization to 78 ms.

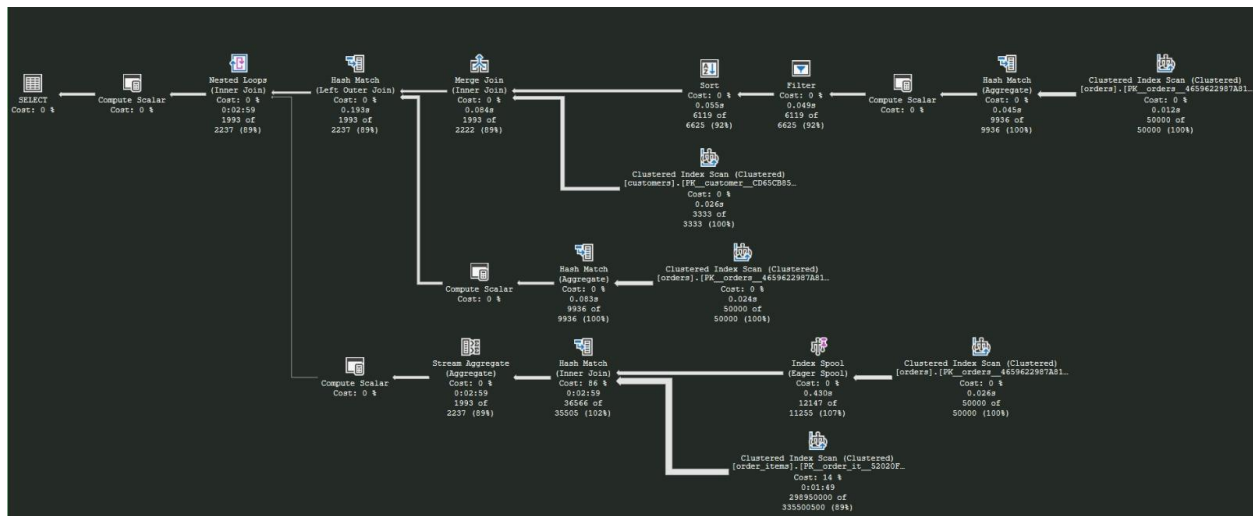


Figure 8 Physical execution plan of the unoptimized query in SSMS, showing the nested loop joins, high-cost table scanning operators, and a intermediate Index Spool (Eager Spool) bottleneck.

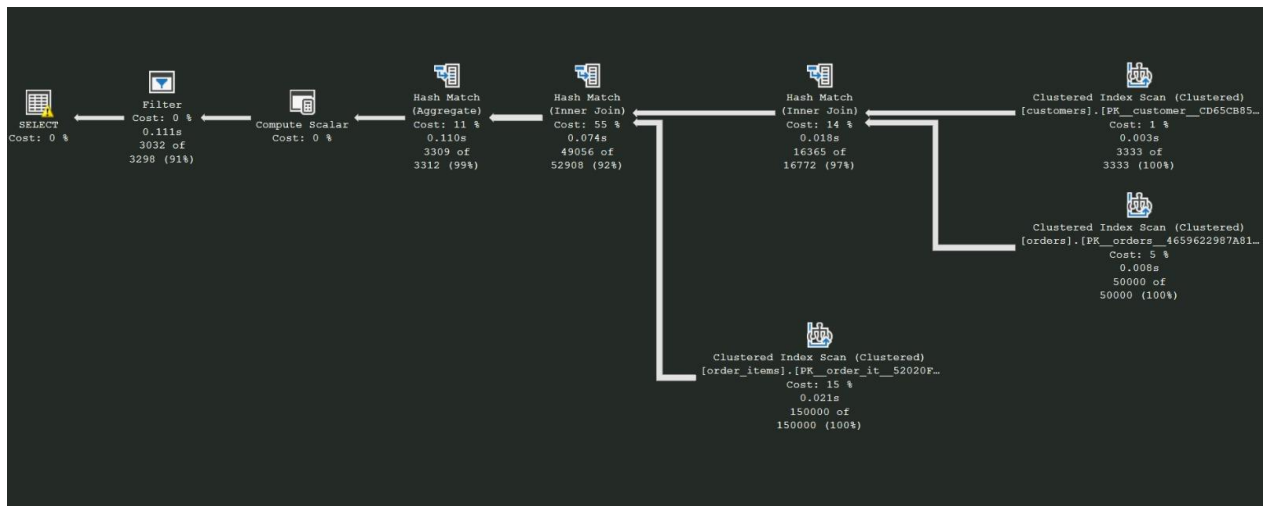


Figure 9 Physical execution plan of the optimized query in SSMS, showing the streamlined set-based Hash Match join and aggregation operators that replaced the nested loops.

4.5 Task 5 Evaluation: Deadlock Lock Analysis

The concurrent transaction simulation demonstrated a deadlock scenario under the locking protocol:

1. Transaction 1 acquired an exclusive lock on Account_A.
2. Transaction 2 acquired an exclusive lock on Account_B.
3. Transaction 2 attempted to lock Account_A and was blocked.
4. Transaction 1 attempted to lock Account_B and was blocked.

The lock manager registered mutual blockages on both transactions, creating a closed cycle in the Waits-For Graph ($T1 \rightarrow T2 \rightarrow T1$). The deadlock detection thread successfully identified the dependency.

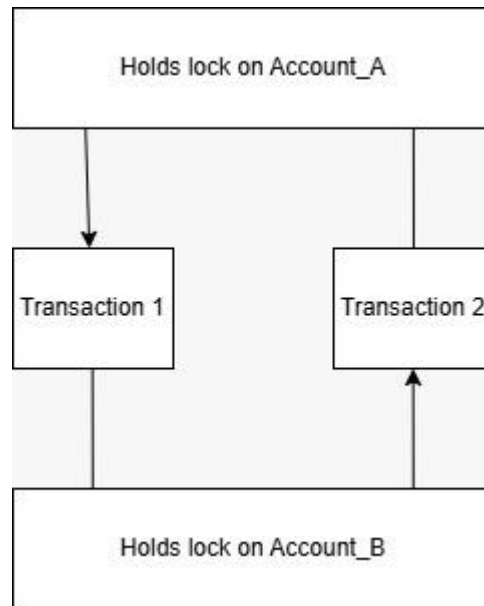


Figure 10 Waits-For Graph (WFG) showing circular locking dependencies and automated transaction rollback

```

Starting Transaction 1
[Lock Acquired] Transaction 1 locked Account_A

Starting Transaction 2
[Lock Acquired] Transaction 2 locked Account_B
[T2 Executing] Attempting to lock Account_A
[T1 Executing] Attempting to lock Account_B
[Lock Conflict] Transaction 1 blocked by Transaction 2 on item 'Account_B'
[Lock Conflict] Transaction 2 blocked by Transaction 1 on item 'Account_A'
[Deadlock Detection/T1 Blocked] Waiting for Account_B
[Deadlock Detection/T2 Blocked] Waiting for Account_A
[Locks Released] Transaction 1 released: ['Account_A']
[Locks Released] Transaction 2 released: ['Account_B']
  
```

Figure 11 Concurrency simulation console output displaying lock acquisition, mutual lock-conflict blocking, deadlock detection, and subsequent rollback/release phases.

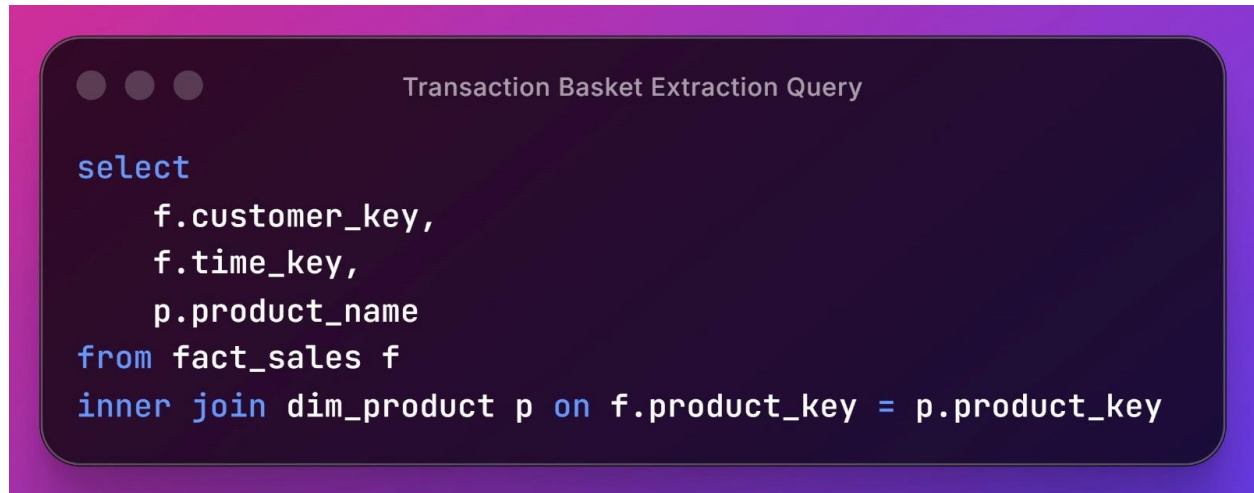
4.6 Task 6 Evaluation: Star Schema Mining Analysis

Our data mining pipeline extracted analytical data directly from the SQL Server Star Schema:

- **Association Mining (Apriori):** One-hot encoding the transaction history from fact_sales and dim_product generated **27 distinct association rules**. Notably, the rule **{Leather Wallet} -> {Charging Cable}** achieved **100% confidence** and a **lift score of 1.50**, indicating a strong cross-selling relationship between these product categories. The rule **{Mechanical**

Keyboard → *Screen Protector* also showed a strong correlation, with **100% confidence** and a **lift score of 1.50**.

- **Customer Classification (Decision Tree):** The model pulled customer demographic data and calculated their total spend. Because all customers currently in the database had spend metrics exceeding \$500 (1,024.0, 1,104.0, and 1,129.0), they were all classified as `is_high_spender = 1`. This uniform distribution resulted in a single leaf node (|--- class: 0), predicting that a new standard-segment customer from the USA would be classified as a high spender (YES).

A screenshot of a code editor window with a dark background and light blue text. The title bar at the top reads "Transaction Basket Extraction Query". The code is a SQL query that selects customer_key, time_key, and product_name from the fact_sales table, joined with the dim_product table on product_key.

```
select
    f.customer_key,
    f.time_key,
    p.product_name
from fact_sales f
inner join dim_product p on f.product_key = p.product_key
```

Figure 12 SQL query executed within the ETL pipeline to retrieve transactional datasets by joining the central fact table with the product dimension table.

A screenshot of a code editor window with a dark background and light blue text. The title bar at the top reads "Customer Profile Extraction Query". The code is a SQL query that selects country, segment, and the sum of sales_amount (aliased as total_spent) from the fact_sales table, joined with the dim_customer table on customer_key, and grouped by customer_key, country, and segment.

```
select
    c.country,
    c.segment,
    sum(f.sales_amount) as total_spent
from fact_sales f
inner join dim_customer c on f.customer_key = c.customer_key
group by f.customer_key, c.country, c.segment
```

Figure 13 SQL query executed within the ETL pipeline to aggregate customer spending metrics and retrieve qualitative demographic attributes from the customer dimension table.

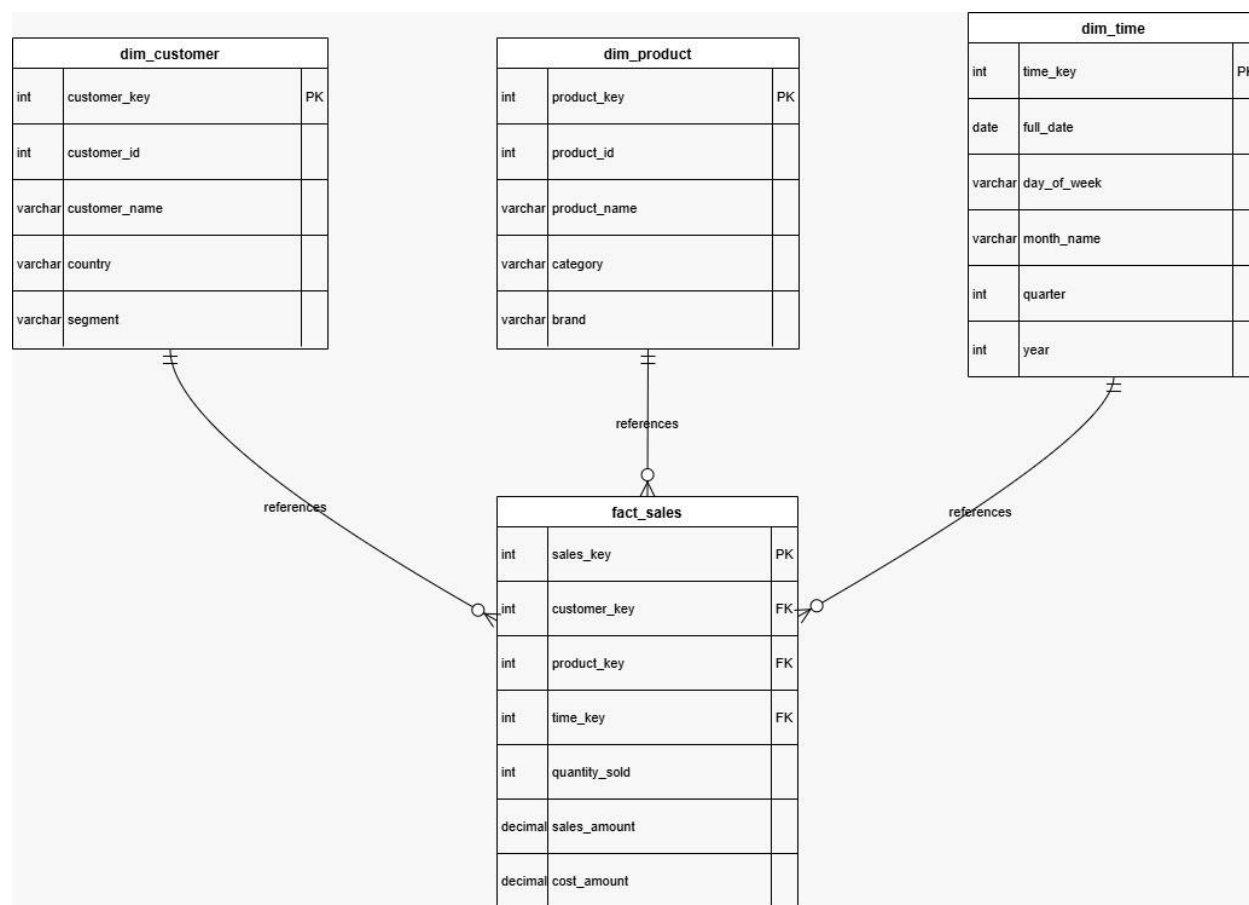


Figure 14 Entity-Relationship Diagram (ERD) of the analytical database, illustrating the de-normalized Star Schema with a central fact table (`fact_sales`) linked to surrounding dimension tables.

Extracted One-Hot Encoded Basket from Star Schema						
product_name	Charging Cable	Leather Wallet	Mechanical Keyboard	Screen Protector	Smartphone X	
transaction_id						
1_1	True	False	False	True	True	
2_1	True	True	False	False	True	
3_2	False	False	True	True	True	

Frequent Itemsets from SQL Server		
support		itemsets
0 0.666667		frozenset({Charging Cable})
1 0.333333		frozenset({Leather Wallet})
2 0.333333		frozenset({Mechanical Keyboard})
3 0.666667		frozenset({Screen Protector})
4 1.000000		frozenset({Smartphone X})
5 0.333333		frozenset({Charging Cable, Leather Wallet})
6 0.333333		frozenset({Charging Cable, Screen Protector})
7 0.666667		frozenset({Charging Cable, Smartphone X})
8 0.333333		frozenset({Leather Wallet, Smartphone X})
9 0.333333		frozenset({Mechanical Keyboard, Screen Protect...})
10 0.333333		frozenset({Mechanical Keyboard, Smartphone X})
11 0.666667		frozenset({Smartphone X, Screen Protector})
12 0.333333		frozenset({Charging Cable, Smartphone X, Leath...})
13 0.333333		frozenset({Charging Cable, Smartphone X, Scree...})
14 0.333333		frozenset({Mechanical Keyboard, Smartphone X, ...})

Generated Association Rules from SQL Server					
	antecedents	consequents	support	confidence	lift
0	frozenset({Charging Cable})	frozenset({Leather Wallet})	0.333333	0.500000	1.50
1	frozenset({Leather Wallet})	frozenset({Charging Cable})	0.333333	1.000000	1.50
2	frozenset({Charging Cable})	frozenset({Screen Protector})	0.333333	0.500000	0.75
3	frozenset({Screen Protector})	frozenset({Charging Cable})	0.333333	0.500000	0.75
4	frozenset({Charging Cable})	frozenset({Smartphone X})	0.666667	1.000000	1.00
5	frozenset({Smartphone X})	frozenset({Charging Cable})	0.666667	0.666667	1.00
6	frozenset({Leather Wallet})	frozenset({Smartphone X})	0.333333	1.000000	1.00
7	frozenset({Mechanical Keyboard})	frozenset({Screen Protector})	0.333333	1.000000	1.50
8	frozenset({Screen Protector})	frozenset({Mechanical Keyboard})	0.333333	0.500000	1.50
9	frozenset({Mechanical Keyboard})	frozenset({Smartphone X})	0.333333	1.000000	1.00
10	frozenset({Smartphone X})	frozenset({Screen Protector})	0.666667	0.666667	1.00
11	frozenset({Screen Protector})	frozenset({Smartphone X})	0.666667	1.000000	1.00
12	frozenset({Charging Cable, Smartphone X})	frozenset({Leather Wallet})	0.333333	0.500000	1.50
13	frozenset({Charging Cable, Leather Wallet})	frozenset({Smartphone X})	0.333333	1.000000	1.00
14	frozenset({Leather Wallet, Smartphone X})	frozenset({Charging Cable})	0.333333	1.000000	1.50
15	frozenset({Charging Cable})	frozenset({Leather Wallet, Smartphone X})	0.333333	0.500000	1.50
16	frozenset({Leather Wallet})	frozenset({Charging Cable, Smartphone X})	0.333333	1.000000	1.50
17	frozenset({Charging Cable, Smartphone X})	frozenset({Screen Protector})	0.333333	0.500000	0.75
18	frozenset({Charging Cable, Screen Protector})	frozenset({Smartphone X})	0.333333	1.000000	1.00
19	frozenset({Smartphone X, Screen Protector})	frozenset({Charging Cable})	0.333333	0.500000	0.75
20	frozenset({Charging Cable})	frozenset({Smartphone X, Screen Protector})	0.333333	0.500000	0.75
21	frozenset({Screen Protector})	frozenset({Charging Cable, Smartphone X})	0.333333	0.500000	0.75
22	frozenset({Mechanical Keyboard, Smartphone X})	frozenset({Screen Protector})	0.333333	1.000000	1.50
23	frozenset({Mechanical Keyboard, Screen Protect...})	frozenset({Smartphone X})	0.333333	1.000000	1.00
24	frozenset({Smartphone X, Screen Protector})	frozenset({Mechanical Keyboard})	0.333333	0.500000	1.50
25	frozenset({Mechanical Keyboard})	frozenset({Smartphone X, Screen Protector})	0.333333	1.000000	1.50
26	frozenset({Screen Protector})	frozenset({Mechanical Keyboard, Smartphone X})	0.333333	0.500000	1.50

Figure 15 Console output of the Apriori data mining execution showing the one-hot encoded matrix, extracted frequent itemsets, and computed association rules (support, confidence, and lift).

```

Extracted Customer Profiles for Classification
  country  segment  total_spent  is_high_spender
0 Pakistan  Premium    1024.0         1
1 Pakistan  Standard    1104.0         1
2 Pakistan  Standard    1129.0         1

--- Generated Decision Tree Rules from Star Schema ---
|--- class: 0

Prediction for standard segment customer from the USA:
-> Will be a High Spender: YES

```

Figure 16 Console output of the Decision Tree classification model showing the aggregated customer profile targets, the single-leaf decision rule, and the premium buyer plan prediction.

5 Technical Challenges & Resolution Strategies

5.1 Task 2 Challenge: Port Binding Conflicts with Default Services

- **Challenge:** When initiating the MongoDB replica set, the command failed with a NoReplicationEnabled error. This occurred because a background Windows service installed by the default MongoDB MSI package was already running on port 27017 and intercepting connections.
- **Resolution:** Shifted the replica set ports to 27018, 27019, and 27020. This bypassed the local service conflict and allowed the instances to initialize properly.

5.2 Task 6 Challenge: SQL Server 'Invalid Object Name' Context Error

- **Challenge:** Executing the Python data mining script threw an Invalid object name 'dim_customer' error, indicating that the table was missing from the target database.
- **Resolution:** This occurred because the table creation commands in SSMS had been executed under the default master database context rather than the oel database context. Prefacing the SQL script with a clear use oel; and go statement ensured the schema was created in the correct database.

This open-ended lab successfully modeled maze solving as an AI search problem and implemented four classical search algorithms in Python. The project satisfies the intended learning outcomes by showing how search problems are represented, how different frontier data structures control behavior, and how heuristic evaluation improves performance in informed search.

The project further extends the classical solver with an interactive visualization layer. This makes the behavior of BFS, DFS, IDS, and A* easier to understand by showing the explored cells, the final path, and the impact of heuristic guidance in a real interface.

5.3 Task 6 Challenge: Pandas SQL Connection Warnings

- **Challenge:** Executing the SQL extraction queries via Pandas triggered UserWarning diagnostics, indicating that pandas performs better when using a SQLAlchemy engine rather than a raw pyodbc connection.
- **Resolution:** Added a warning suppression filter using Python's warnings library (`warnings.filterwarnings('ignore', category=UserWarning)`), preserving clean console outputs for the final report.

6 Conclusion & Future Work

This paper presents a comprehensive, task-by-task empirical evaluation of modern database engineering principles across six key areas. The experimental results demonstrate how distributed consistency, indexing paradigms, physical execution plan operators, transaction isolation, and dimensional analytics impact overall database performance and availability.

Future work includes:

- Integrating *Vector Clocks* to resolve data divergence in AP systems without relying on physical system clocks.
- Replacing the Apriori algorithm with the *FP-Growth (Frequent Pattern Growth)* algorithm to speed up pattern extraction by avoiding resource-heavy candidate generation steps.
- Upgrading the single Decision Tree classifier to a *Random Forest or Gradient Boosting* ensemble model to reduce overfitting and improve prediction accuracy on larger, more complex datasets.